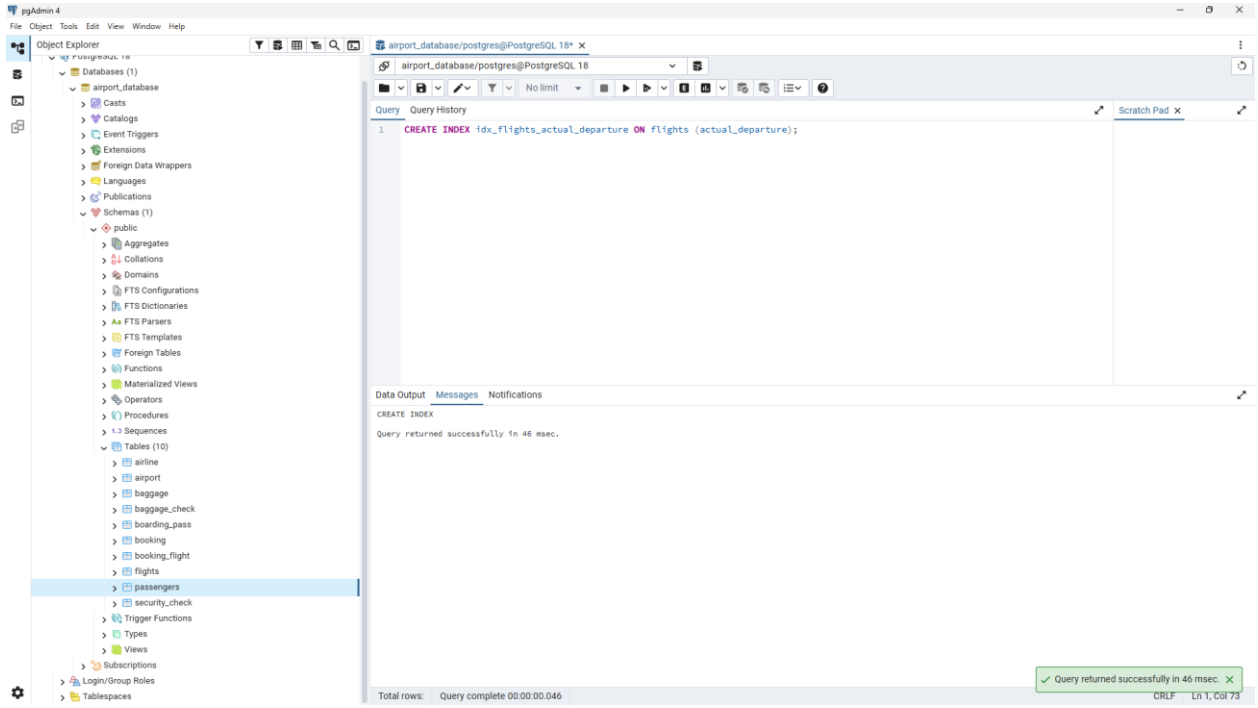
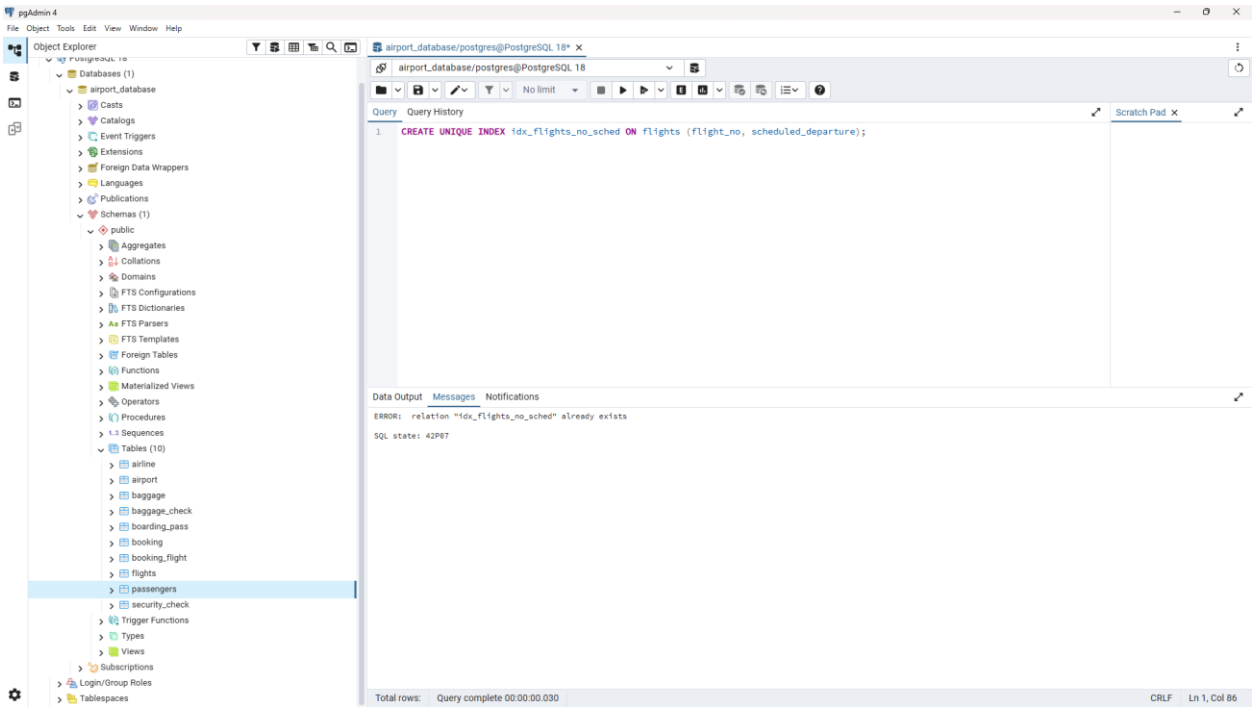
Laboratory Work 7

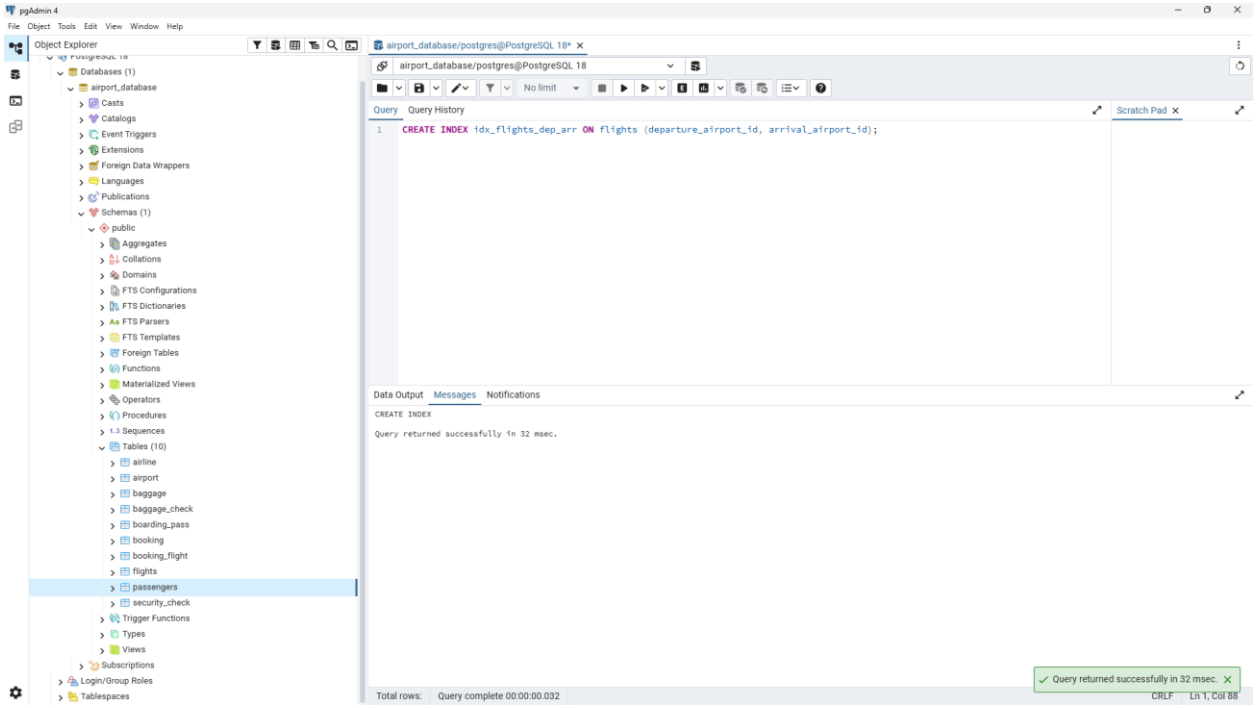
1. Create an index on the actual_departure column in the flights table.



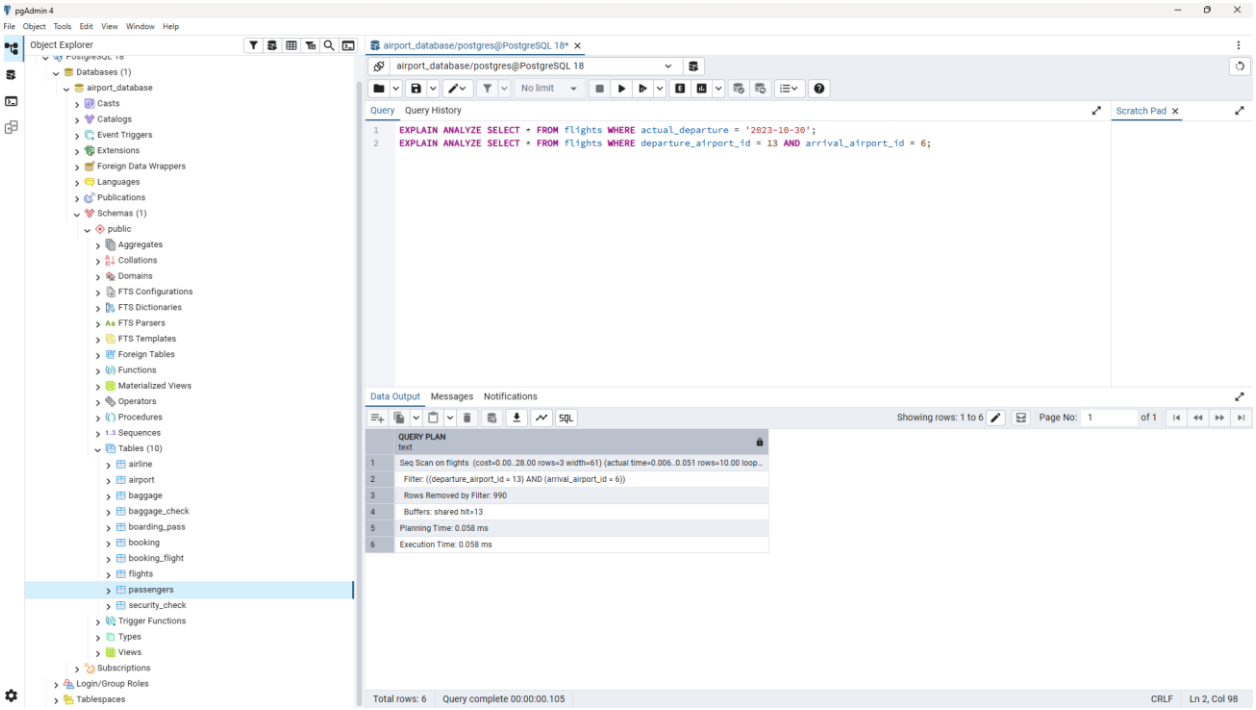
2. Create a unique index to ensure flight_no and scheduled_departure combinations are unique.

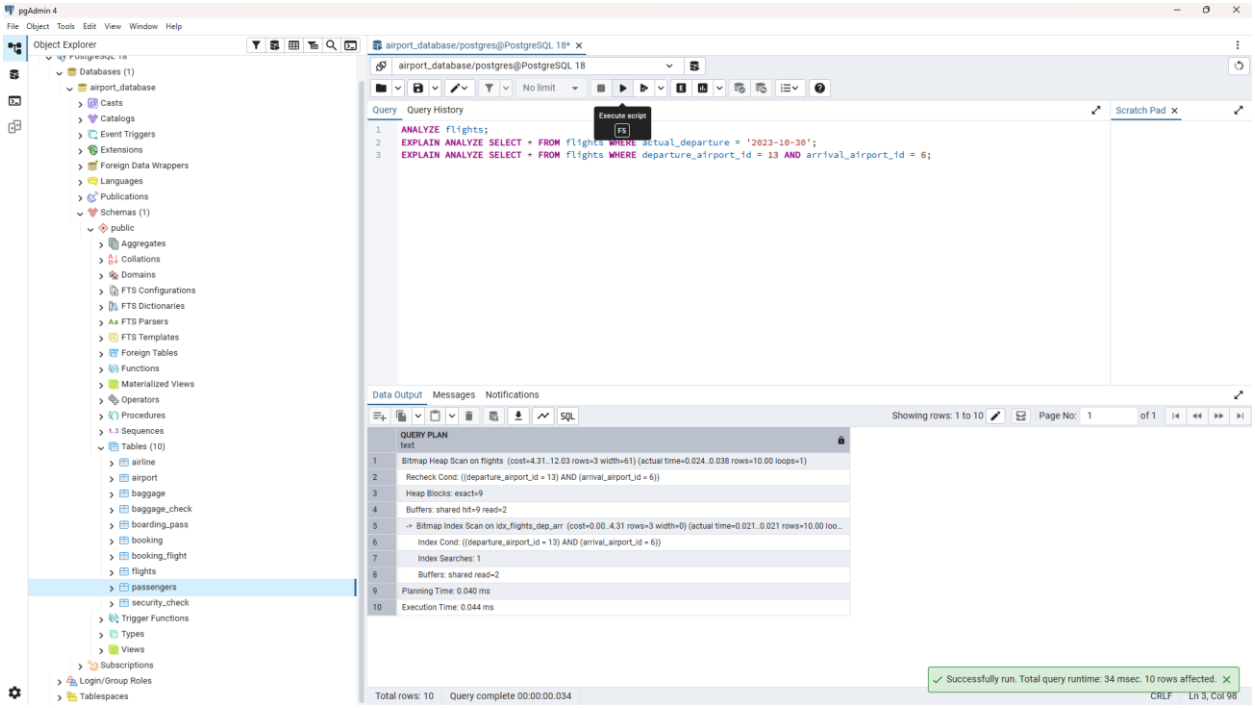


3. Create a composite index on the departure_airport_id and arrival_airport_id columns.

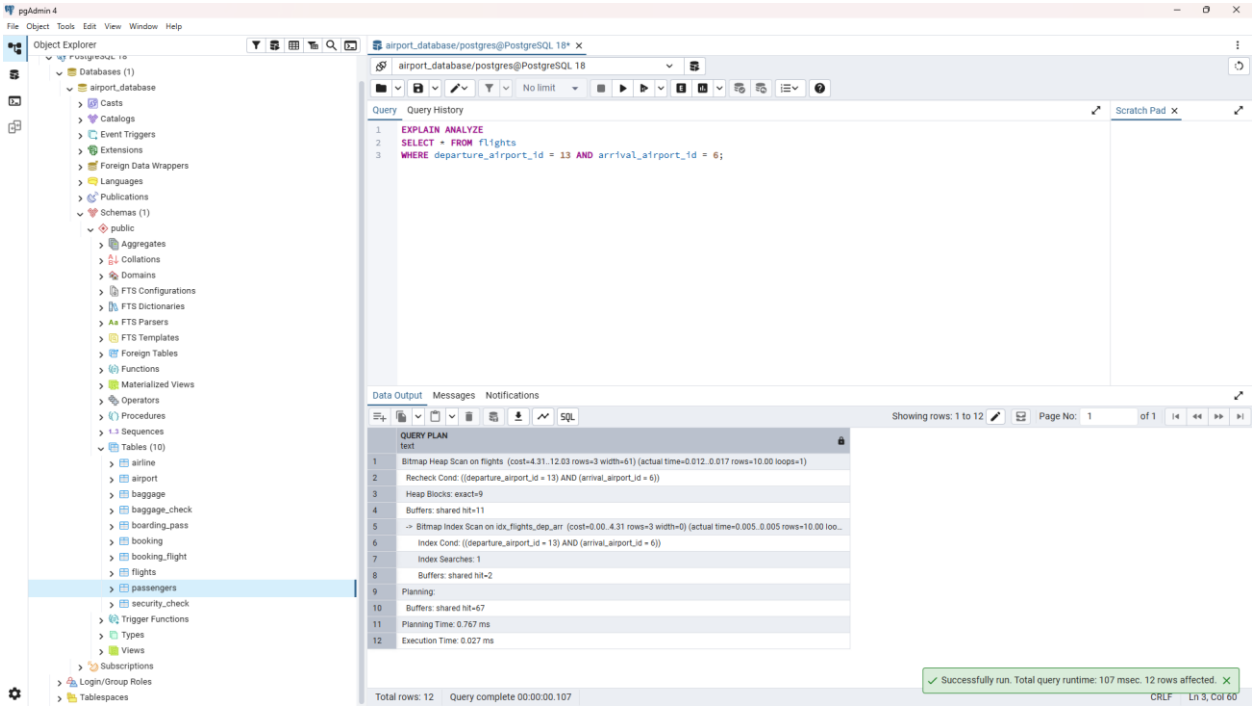


4. Evaluate the difference in query performance with and without indexes. Measure performance differences.

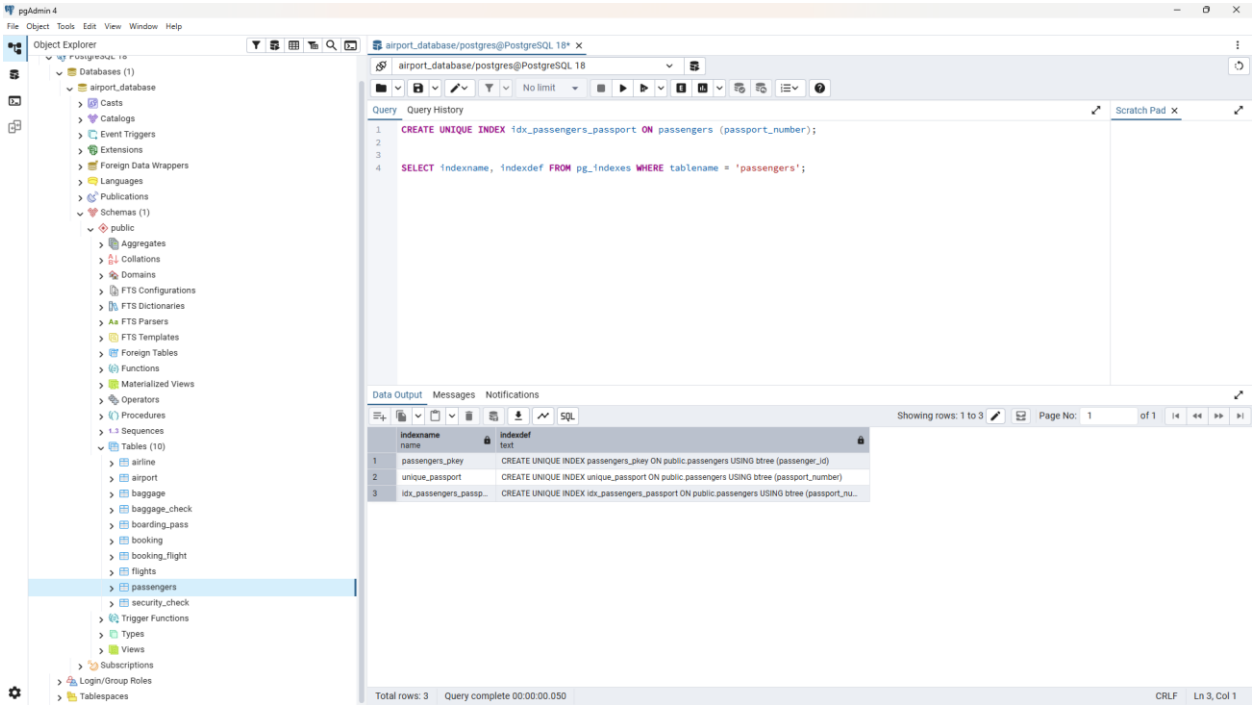




5. Use EXPLAIN ANALYZE to check index usage in a query filtering by departure_airport and arrival_airport.



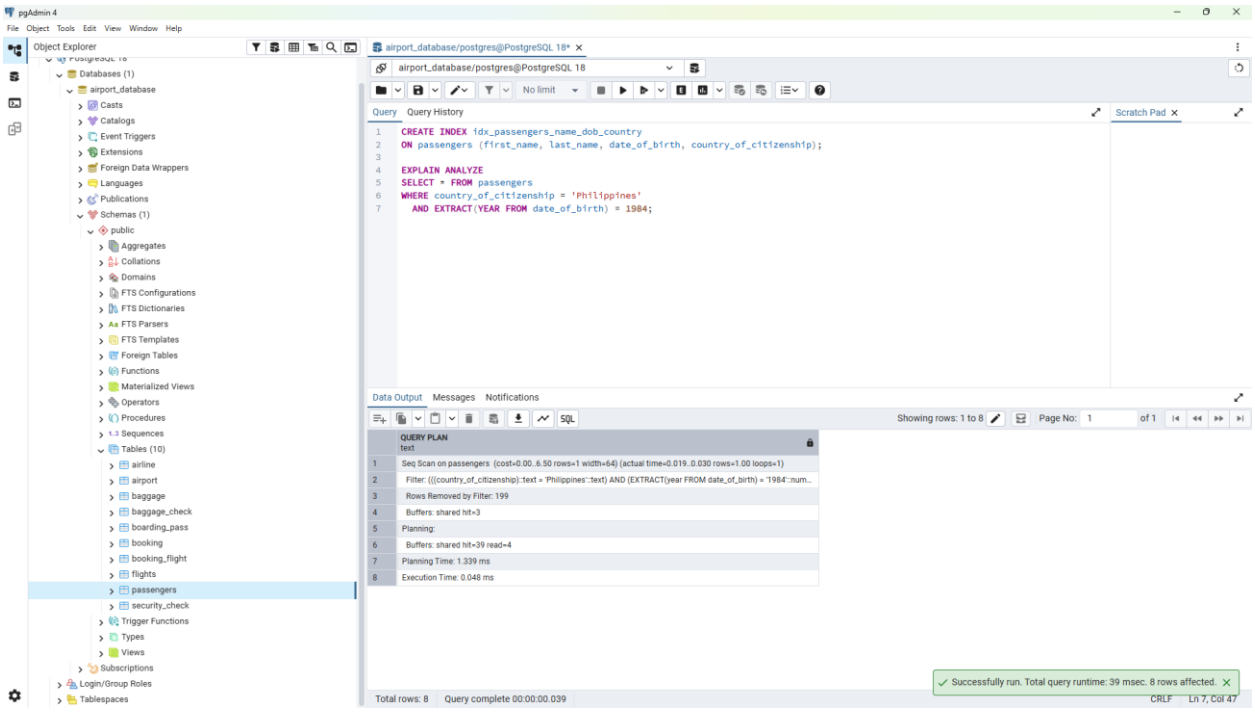
6. Create a unique index for the passport_number of the Passengers table. Check if the index was created or not. Insert into the table two new passengers.



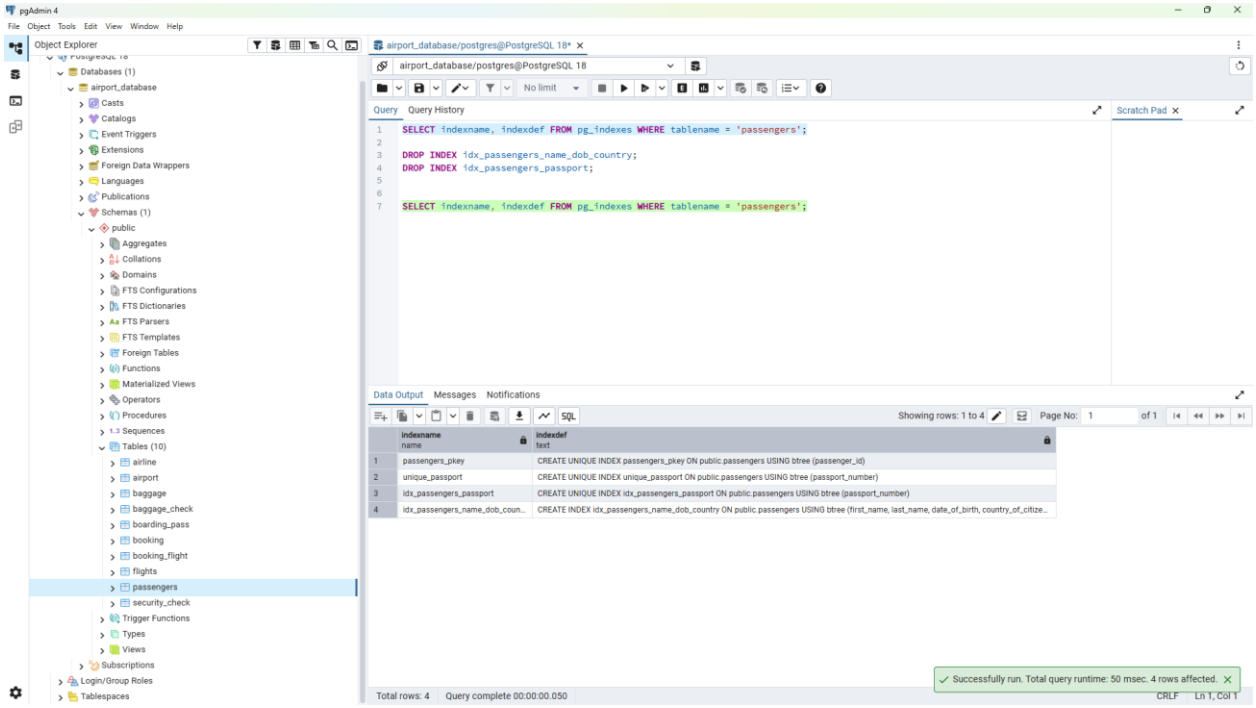
Explain in your own words what is going on in the output?

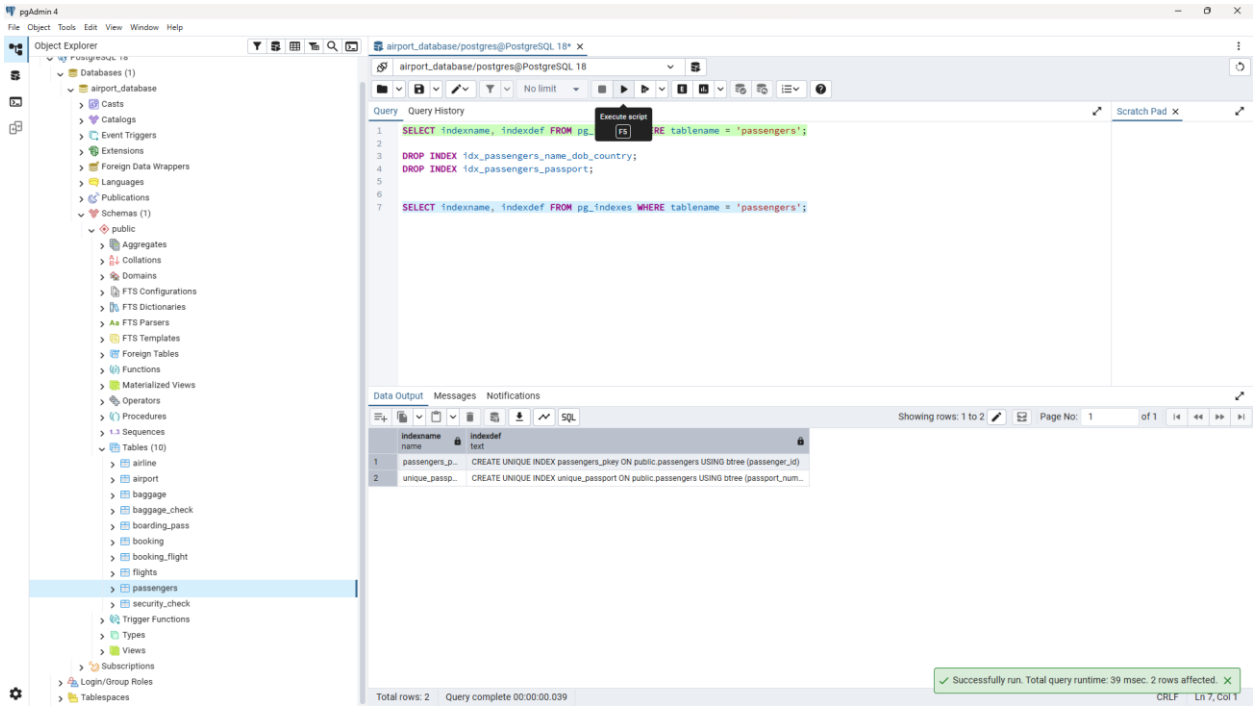
A unique index is a rule, not only a speed-up. The first passenger with that passport is accepted. The second one has the same passport number, and the database refuses it so that two people can never share one passport. Passenger 201 stays in the table, 202 is not added.

7. Create an index for the Passengers table. Use for that first name, last name, date of birth and country of citizenship. Then, write a SQL query to find a passenger who was born in Philippines and was born in 1984 and check if the query uses indexes or not. Give the explanation of the results.

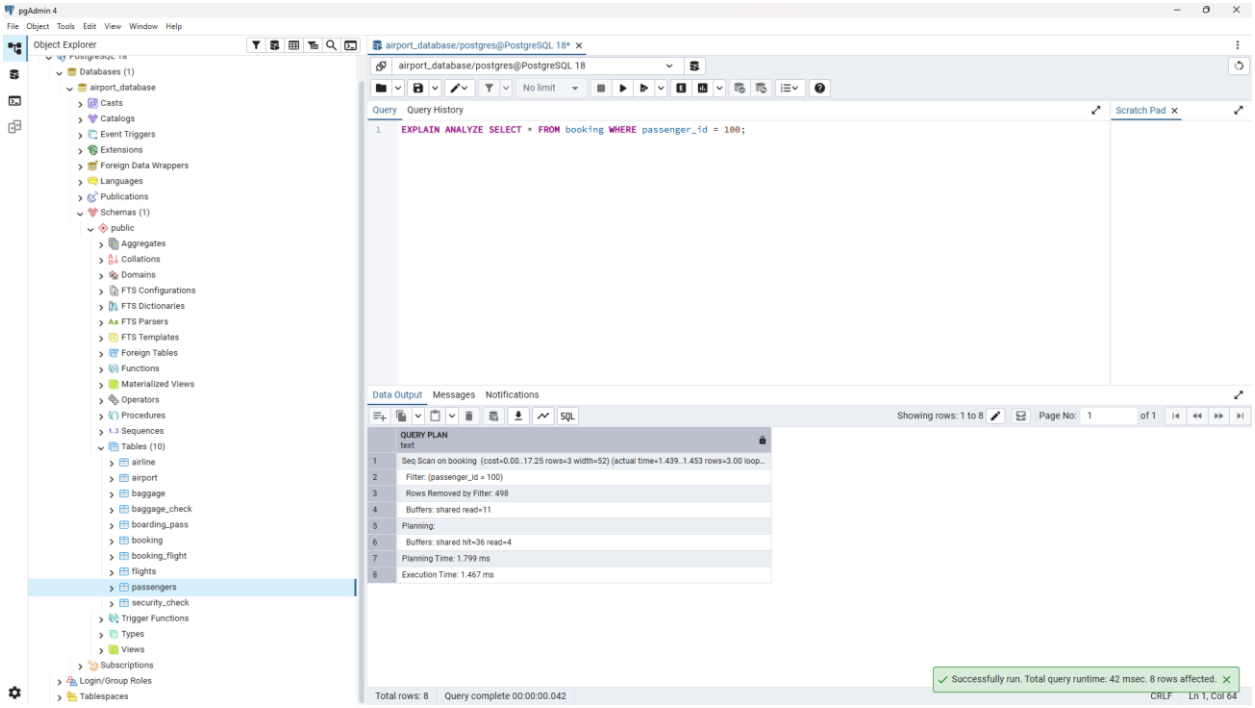


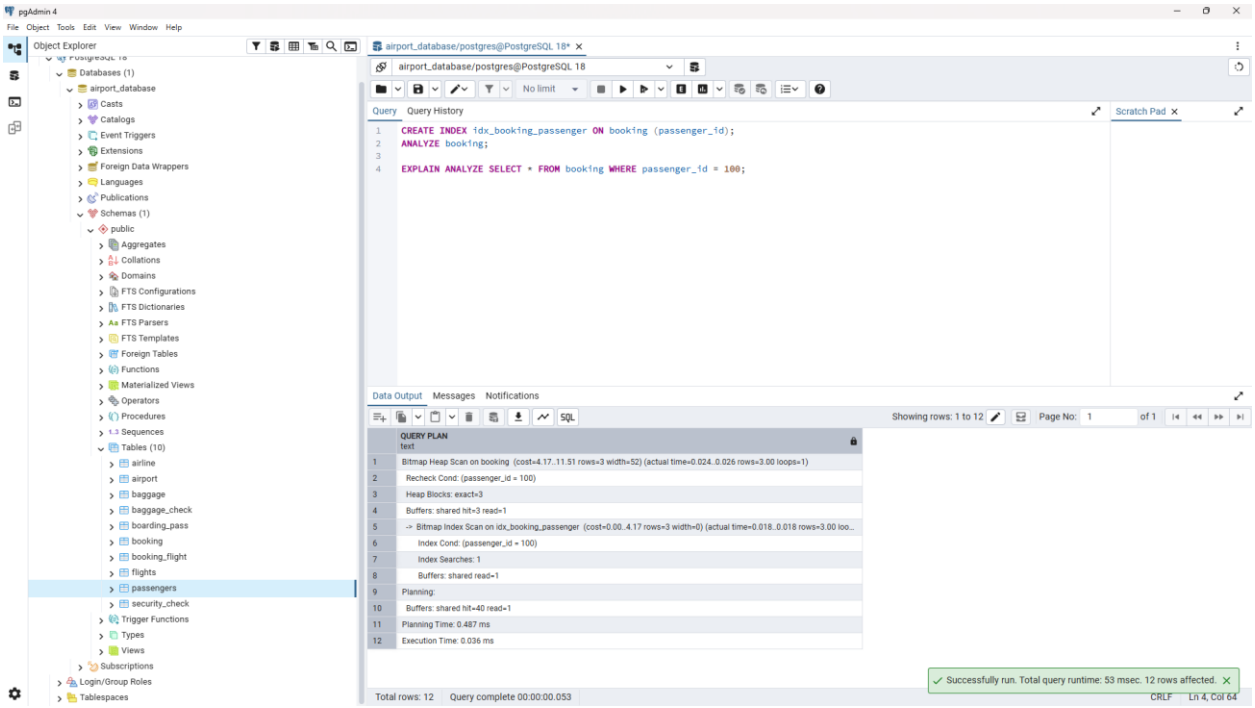
8. Write a SQL query to list indexes for table Passengers. After delete the created indexes.





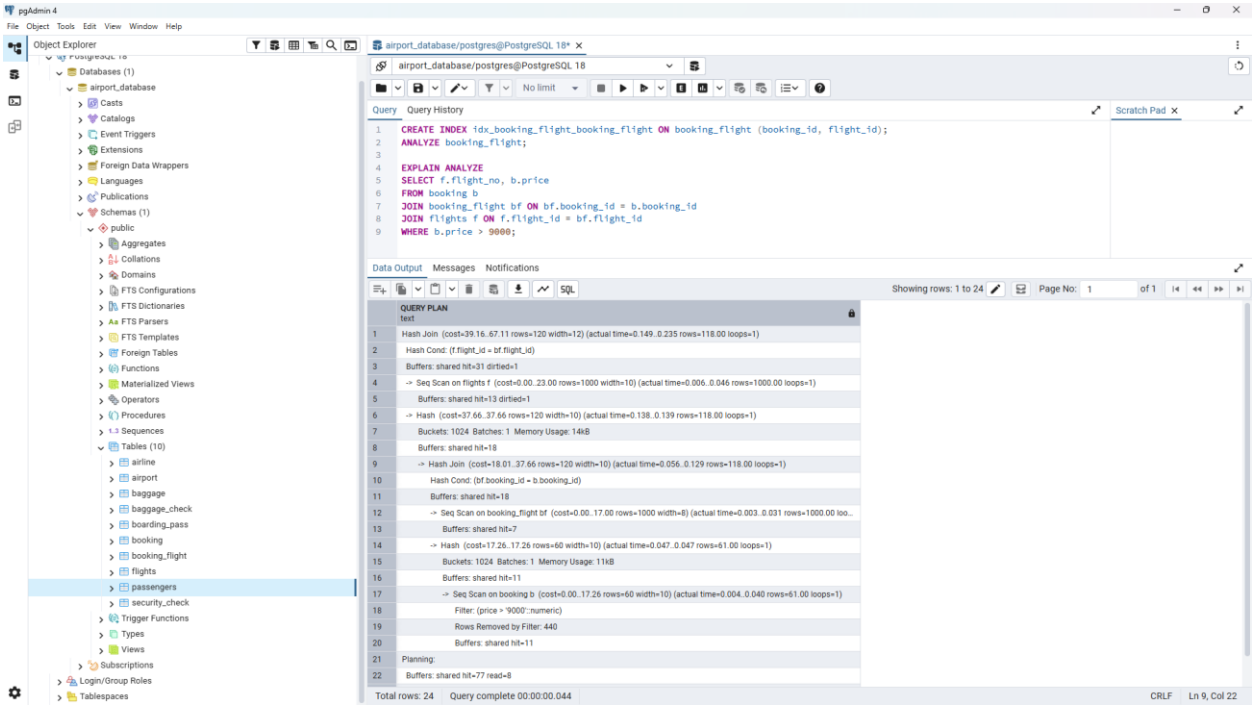
9. Create an index on the passenger_id column of the booking table. Using EXPLAIN ANALYZE, compare the execution plan and execution time of a query that selects all bookings of one passenger (for example, passenger_id = 100) before and after creating the index. Explain what changed in the plan.



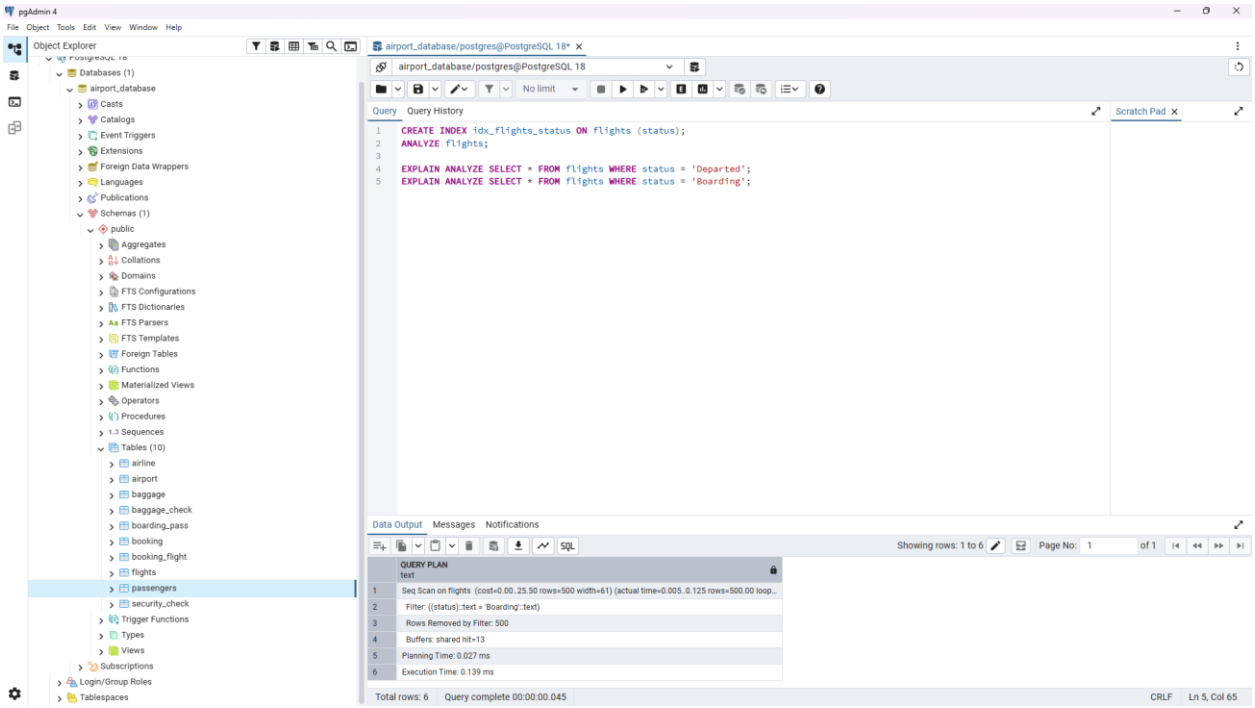


Before, the database looked at all 500 bookings and threw away 497 of them. After, it jumps to the 3 matching rows through the index. The plan changed from Seq Scan to an index scan, and the time dropped.

10. Create a composite index on the booking_flight table using the booking_id and flight_id columns. Write a query that joins booking, booking_flight and flights and returns the flight_no and price for bookings with a price greater than 9000. Check with EXPLAIN ANALYZE which indexes are used in each step of the join.

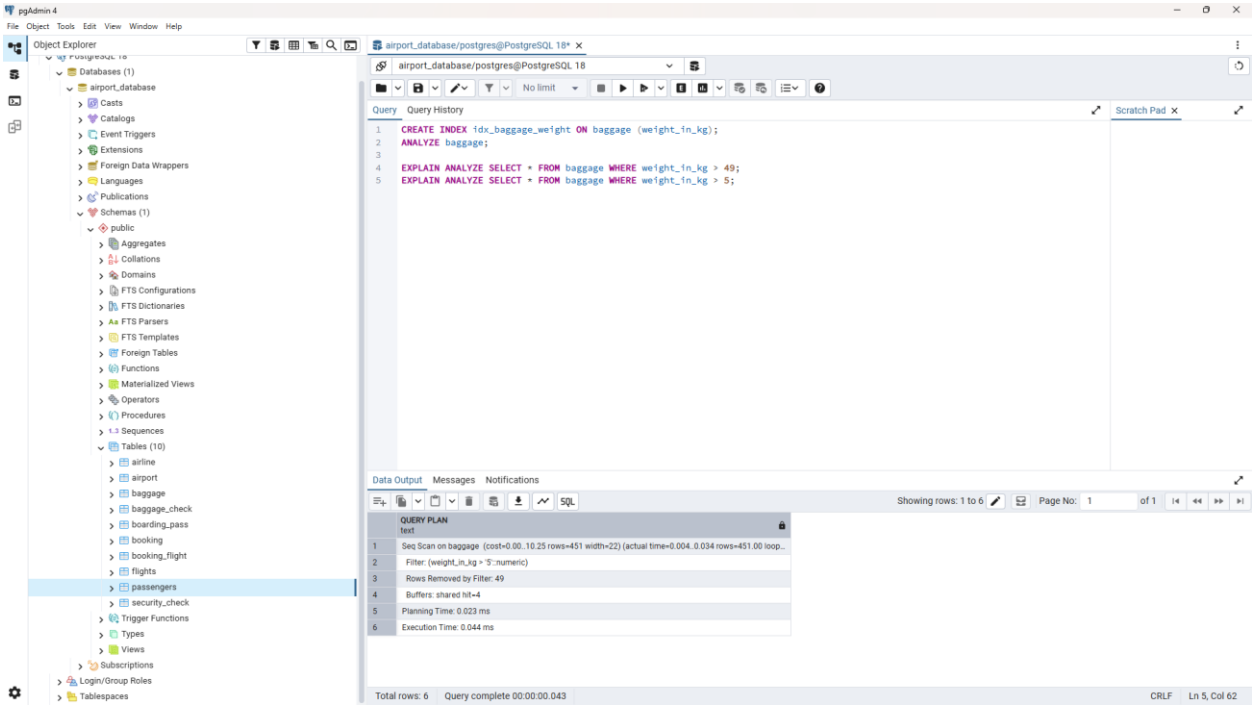


11. Create an index on the status column of the flights table. Run EXPLAIN ANALYZE for the query with status = 'Departed' and for the query with status = 'Boarding'. Compare the plans and explain, in your own words, why the optimizer may ignore the index for one of the values (hint: selectivity).



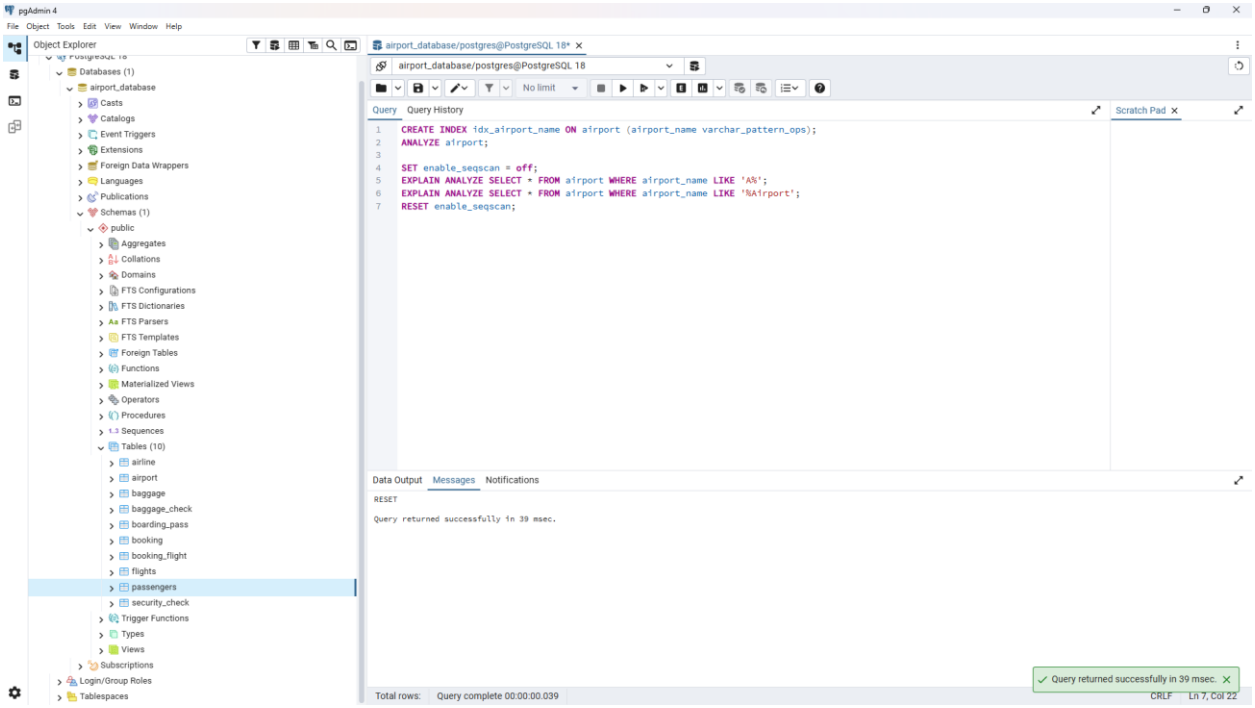
Selectivity means "how few rows match". 'Departed' matches almost nothing, so the index is a great shortcut. 'Boarding' matches half the table. Using the index would mean 500 lookups and 500 jumps into the table, which is more work than just reading the whole table once. The optimizer knows this from its statistics and ignores the index.

12. Create an index on the weight_in_kg column of the baggage table. Compare the plans of two range queries: weight_in_kg > 49 (a small part of the table) and weight_in_kg > 5 (almost the whole table). Explain the difference in index usage.



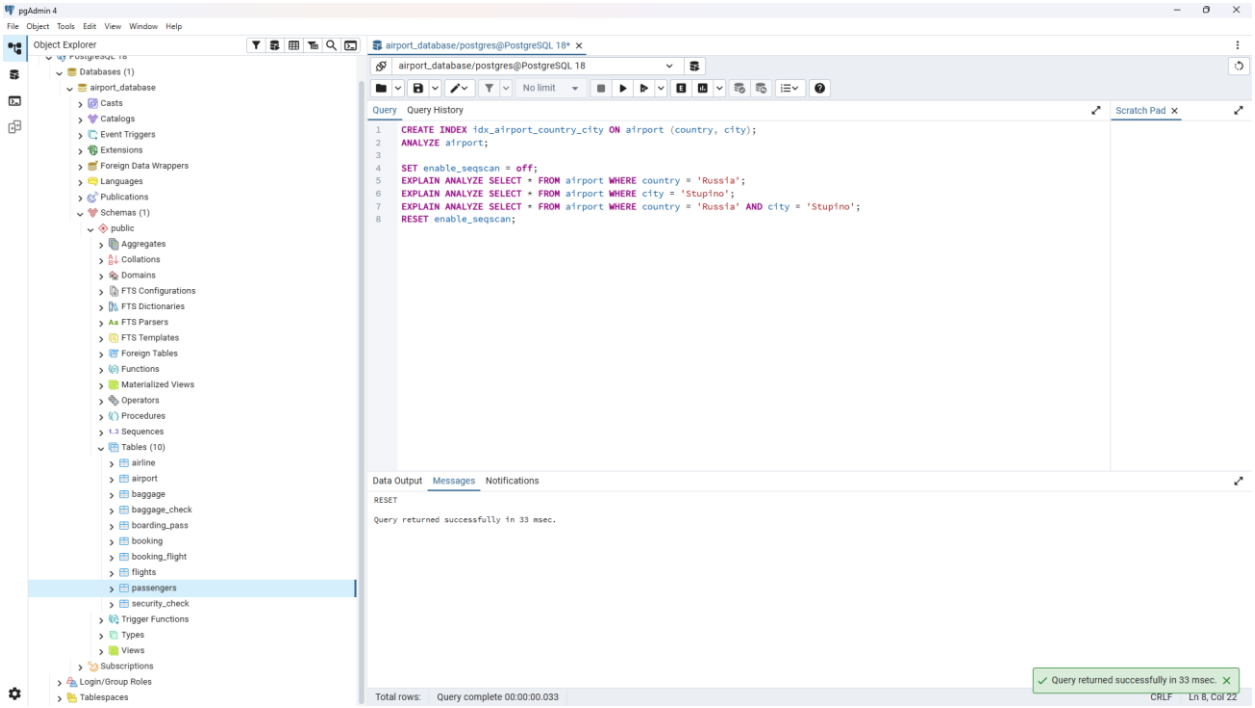
The same idea as Task 11. For > 49 the index quickly finds the few heavy bags. For > 5 almost every row qualifies, so going through the index first is a waste and reading the whole table is cheaper.

13. Create an index on the airport_name column of the airport table. Check with EXPLAIN ANALYZE whether it is used by the queries LIKE 'A%' and LIKE '%Airport'. Explain why the results are different.



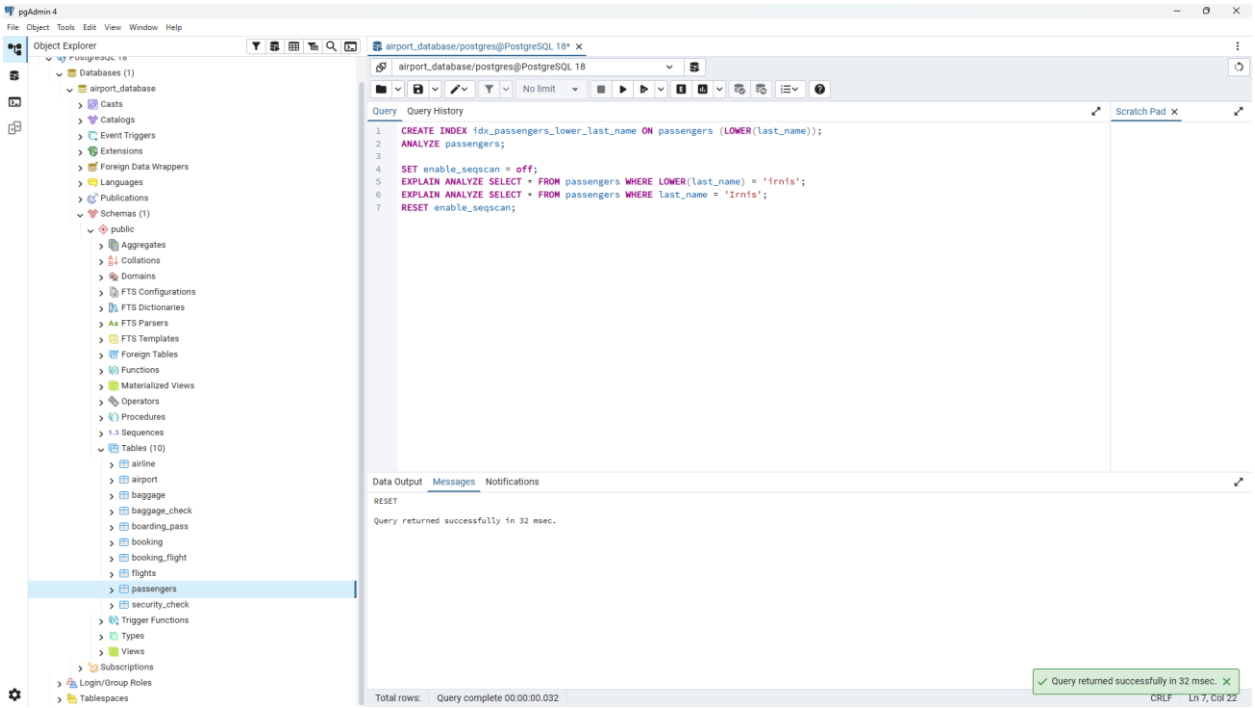
The index is sorted alphabetically from the first letter. "Starts with A" means the database can jump to the A section and stop. "Ends with Airport" could match a name that starts with any letter, so there is no place to jump to and every row must be checked.

14. Create a composite index on the country and city columns of the airport table. Run three queries: filtering by country only, by city only, and by both columns. Check which of them use the index and explain the leftmost prefix rule using your results.



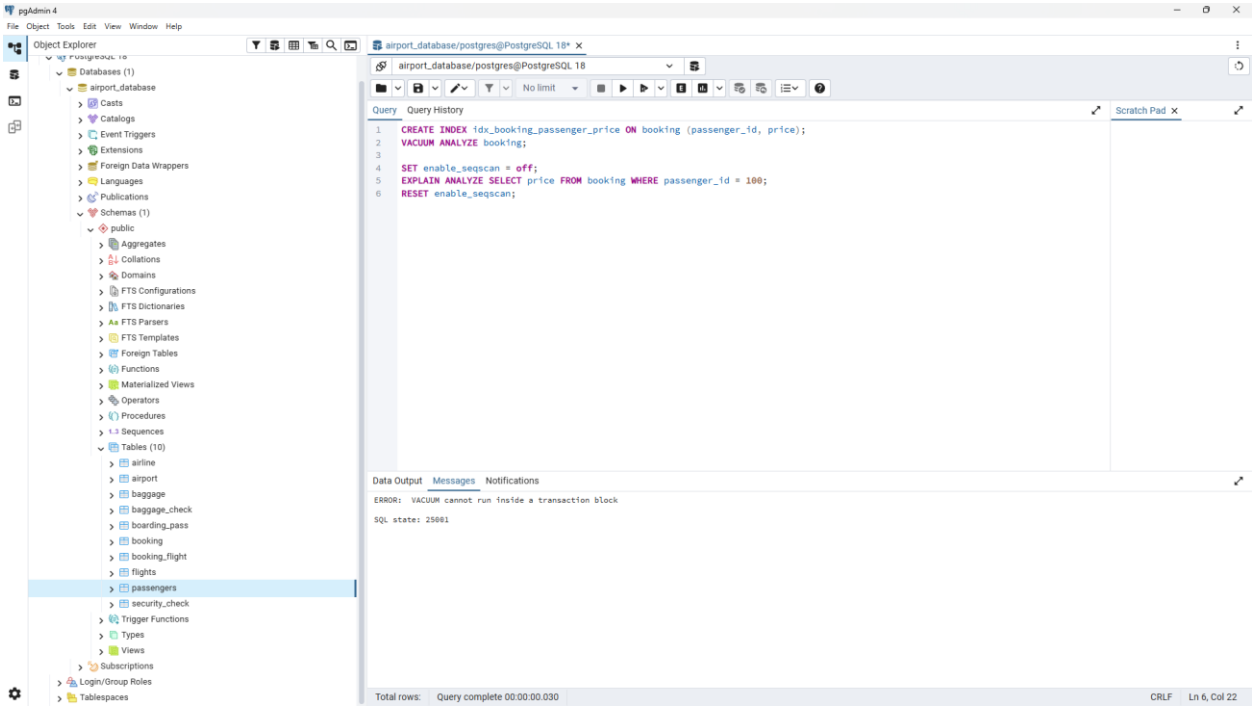
The leftmost prefix rule says a multi-column index only works fast when the query starts with the first column of the index (then the second, and so on). It is like a phone book sorted by country and then city: you can look up a country, or a country plus a city, but you cannot quickly find a city alone because the same city name is scattered under every country.

15. Create an expression (functional) index on the lower-case last_name of the Passengers table (LOWER(last_name)). Compare the plans of a query written as WHERE LOWER(last_name) = 'irnis' and a query written as WHERE last_name = 'Irnis'. Explain which query can use the index and why.



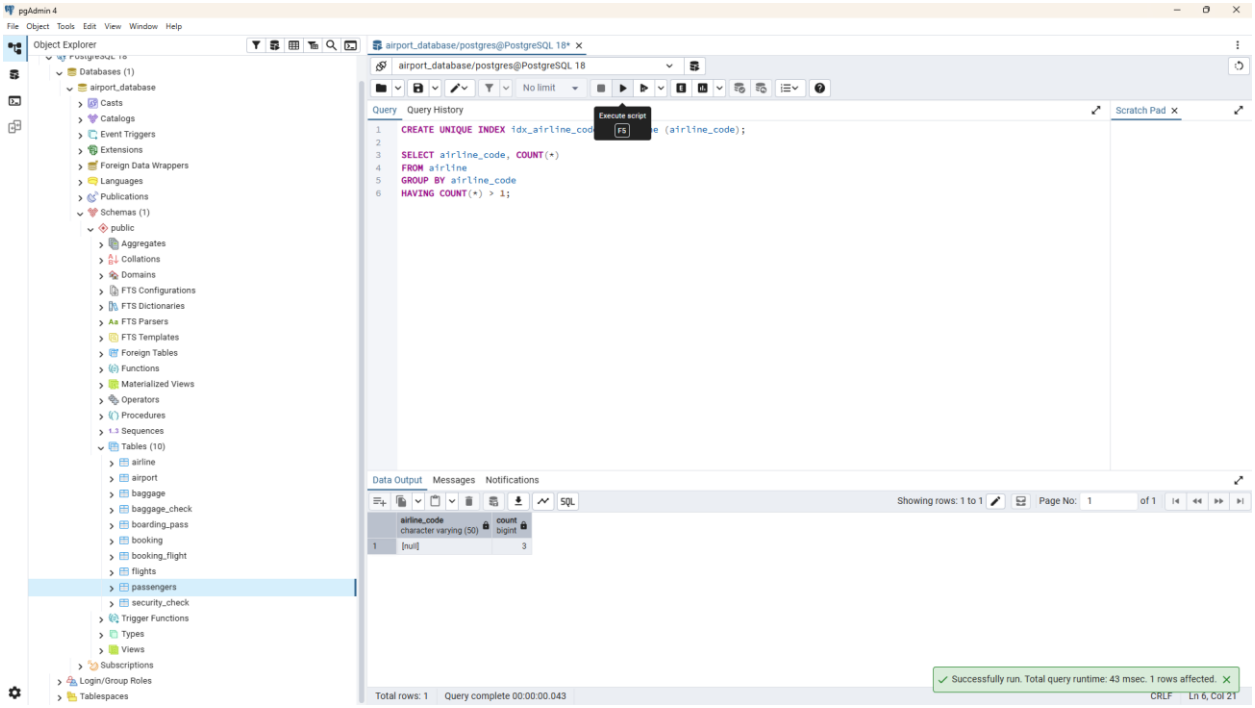
This index stores the lower-case version of every last name. The first query asks for exactly that, LOWER(last_name), so it matches the index. The second query asks for plain last_name, which is a different thing from what is stored, so the database cannot use the index. The rule: the expression in the query must be the same as the expression in the index.

16. Create a covering index on the booking table for the columns passenger_id and price. Write a query that returns only price for a given passenger_id and check with EXPLAIN ANALYZE whether an index-only scan (or "Using index") is used. Explain the benefit of a covering index.



The index already contains both columns the query needs, passenger_id to find the rows and price to return. So the database answers the question from the index alone and never opens the table. That saves reading from the table, which makes the query faster. In MySQL the same thing shows as "Using index".

17. Try to create a unique index on the airline_code column of the airline table. If an error occurs, explain what caused it and find the problematic rows with a GROUP BY ... HAVING COUNT(*) > 1 query. Then create the unique index in a way that works for the current data, or after fixing the data.



NULL means "unknown", and the database does not treat two unknowns as equal, so a unique index allows many NULLs. The 3 empty codes are therefore not duplicates.